10

Лекция 13–14 апреля

Универсальная машина

Константин Чухарев

Дискретная математика — Весна 2027

Варианты и устойчивость

> *Абстракция нужна не для тумана, а для нового семантического уровня, на котором можно быть абсолютно точным.*
>
> ***— Эдсгер Дейкстра***

Произвол в определении

В семёрке машины легко заподозрить произвол.

Таблица показывает, что каждую альтернативу обсуждали всерьёз.

В определении	**Разумная альтернатива**
лента бесконечна в обе стороны	лента бесконечна только вправо
сдвиг ровно на клетку	остаться на месте
одна лента и одна головка	k лент с отдельными головками
детерминированный переход	недетерминированный выбор перехода

Если бы класс вычислимого зависел от этих деталей, слово «вычислимо» описывало бы одну конструкцию, а не понятие.

Поэтому эквивалентность вариантов проверяют прежде, чем строить на модели теорию.

Многоленточная машина

Программисту привычны несколько массивов данных, и k лент выглядят как удобство, а не как новая сила.

Определение 1 (Многоленточная МТ)

В машине k лент и k головок.

За один шаг каждая головка читает, пишет и сдвигается независимо.

Симуляция выкладывает все ленты на одну колонками. В колонке идут первые клетки всех лент, за ними вторые, а клетку под головкой метит шляпка.

Две ленты на одной

Упаковка видна на паре коротких лент.

Пример (Упаковка по колонкам)

Первая лента содержит 011, вторая — 10, головки стоят на первых клетках.

	1	**2**	**3**
лента 1	$\hat{0}$	1	1
лента 2	1	$\hat{0}$	$\square$

Шляпки показывают обе головки сразу. Один шаг исходной машины читает символы под шляпками, находит переходы, записывает новое и сдвигает шляпки.

Упаковка по колонкам — это **дорожки**. Ячейка хранит кортеж символов, по одному на ленту, а алфавит общей ленты — произведение алфавитов. Произведение конечных алфавитов конечно, поэтому машина с дорожками остаётся машиной Тьюринга.

Копирование на двух лентах

Вторая лента покупает не силу, а шаги.

Пример (Один проход и челнок)

Копирование слова длины n на вторую ленту — один проход, то есть n шагов.

На одной ленте копия строится челночными пробегами между источником и копией, и каждый новый символ требует прохода через всё слово. Шагов выходит порядка n^2.

Обе версии вычисляют одно и то же, и разница только в числе шагов.

Цена симуляции

Симуляция честная, она повторяет каждый шаг исходной машины. Точность обходится временем.

За t шагов каждая головка уходит от старта не дальше чем на t клеток, поэтому занятая часть общей ленты растёт линейно. При $T = 1000$ шагов она занимает не больше 2001 клетки, и обход её на каждом шаге даёт около $2 \cdot 10^6$ шагов симулятора.

После $T(n)$ шагов многоленточной машины занятая часть занимает $O(T(n))$ клеток.

Каждый шаг симуляции обходит её целиком.

В сумме симуляция тратит $O(T(n)^2)$ шагов.

Недетерминированная машина

Некоторые задачи естественно описываются угадыванием.

Определение 2 (НМТ)

Функция переходов возвращает множество вариантов.

$$\delta : Q \times \Gamma \to \mathcal{P}(Q \times \Gamma \times \{L, R\}).$$

Машина принимает вход, если хотя бы одна ветвь вычисления достигает q_{accept}.

Ветвление — единственное отличие. Детерминированная машина берёт один переход из пары «состояние и символ», недетерминированная — целое множество.

Угадывание делителя

Пара чисел показывает, как работает перебор ветвей.

Пример (Составное число и простое)

Машина, распознающая составные числа, получает n и угадывает делитель. Ветви перебирают $d = 2, 3, \ldots, n - 1$ и делят n на d нацело.

На входе 91 ветвь с $d = 7$ получает частное 13 и проверяет $7 \cdot 13 = 91$. Произведение совпадает со входом, и эта ветвь принимает.

На простом входе 89 перебираются 87 ветвей. Ни одна не даёт целого частного, и принятия нет ни в одной ветви.

Принять достаточно одной ветви, поэтому квантор существования сидит в определении принятия.

Почему обход в ширину?

Детерминированная симуляция НМТ обходит дерево конфигураций в ширину. Все ветви продвигаются по одному шагу.

Глубинный обход рискует навсегда уйти в бесконечную ветвь и пропустить принимающую, которая сидит в соседней.

Ширина дерева ограничена.

Из вершины выходит не более b переходов, поэтому на глубине t стоит не более b^t вершин.

При $b = 2$ и глубине 10 обход держит в работе $2^{10} = 1024$ ветви прежде, чем пойти на глубину 11.

Плата — экспонента, но для вопроса «принимает ли» важна возможность, а не скорость.

Устойчивость модели

Четыре строки таблицы — четыре кандидата в определение. Каждый из них симулируется базовой машиной.

Теорема 1 (Эквивалентность вариантов)

Односторонняя лента, сдвиг на месте, k лент и недетерминизм вычисляют тот же класс языков, что и одноленточная детерминированная МТ.

То же верно для машин с двумя стеками, машин Минского с двумя счётчиками и регистровых машин.

Симуляция вариантов

Доказательство

Докажем симуляцией, построив для каждой модификации повторяющую её машину.

Односторонняя лента. Правая половина ленты идёт по верхней дорожке, левая — отражённой по нижней, а границу метит нулевая клетка. Переход через границу меняет дорожку, и шаг исходной машины стоит постоянного числа шагов симулятора.

Сдвиг на месте. Правило $(q, a) \to (q', b, S)$ заменяется парой через свежее состояние r.

$$(q, a) \to (r, b, L), \quad (r, x) \to (q', x, R) \text{ для всех } x \in \Gamma.$$

Два шага симулятора дают один шаг исходной машины.

Симуляция лент и ветвления

Доказательство (продолжение)

Несколько лент. Ленты выкладываются колонками с метками головок. Один шаг исходной машины стоит нескольких проходов по занятой части, и время такой симуляции квадратично.

Стеки и счётчики. Стек — участок ленты с вершиной под головкой, а два стека лежат по разные стороны головки. Счётчик — число в унарной записи, регистры регистровой машины — отдельные участки ленты.

Недетерминизм. Дерево конфигураций обходится в ширину, поэтому бесконечная ветвь не мешает найти принимающую.

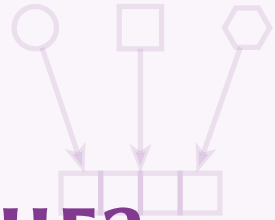
Что доказывает эквивалентность?

Сила модели определяется задачами, а не деталями конструкции.

О вычислимости можно говорить, не уточняя версию машины.

Дальше слово «машина» свободно означает любую из версий.

Доказательства ведутся на той, где меньше техники, а вывод переносится на все остальные.



Тезис Чёрча–Тьюринга

> *Впервые удалось дать абсолютное определение интересного эпистемологического понятия — понятия, не зависящего от выбранного формализма.*
>
> **— Курт Гёдель**

Вычислимая функция

Ответ машины не ограничен словом «да» или «нет». На ленте может остаться новое слово.

Определение 3 (Вычисление функции)

МТ **вычисляет** функцию $f : \Sigma^* \hookrightarrow \Sigma^*$, если на входе w она останавливается с $f(w)$ на ленте в точности тогда, когда $f(w)$ определено.

Функция может быть частичной. Там, где функция не определена, машина не останавливается вовсе.

Функция, определённая на всех словах, называется всюду определённой (*total*).

Частичность на машинах

Различие проще увидеть на двух машинах, чем запомнить по определению.

Пример (Всюду определённая и частичная)

Инверсия битов вычисляет всюду определённую функцию. Машина проходит строку до пробела и останавливается, так что $f(001) = 110$.

Половина унарной длины — частичная функция. Машина вычёркивает в источнике по две единицы и дописывает одну единицу в результат.

На 1^6 набирается три пары, и на ленте остаётся 1^3. На 1^5 после двух пар остаётся одна единица без пары, машина не находит пары и зацикливается, а $f(1^5)$ не определено.

Частичность — отсутствие остановки, а не особый ответ машины.

На нечётной длине машина не сообщает ошибку, она просто не останавливается.

Что утверждает тезис?

Определение вычислимой функции выбрано, но выбор требует обоснования. Нужно объяснить, почему мера вычислимого — именно машина.

Тезис Чёрча–Тьюринга
Класс функций, вычислимых на машине Тьюринга, совпадает с классом интуитивно вычислимых функций.

Класс интуитивно вычислимых функций формального определения не имеет, поэтому совпадение проверяют опытом.

Из тезиса следует правило для практики. Если задача неразрешима на МТ, она неразрешима нигде. Ни на другом языке, ни на другой архитектуре, ни на суперкомпьютере она не станет разрешимой.

Чем тезис не является?

Тезис — не теорема.

Теорема связывает два формальных объекта, а «интуитивно вычислимая функция» формальна не больше, чем наши представления о любом конечном процессе.

Статус у тезиса другой. Это принятое определение понятия «алгоритм», проверяемое опытом.

Проверка идёт с двух сторон.

Все независимые формализмы сходятся к одному классу, а процедура, вычисляющая что-то вне его, за девяносто лет не предъявлена.

Три формализма 1936 года

Одна и та же граница вычислимого найдена трижды и независимо.

Формализм	Чем торгует
Машина Тьюринга	лента, головка, состояния
Лямбда-исчисление Чёрча	подстановка вместо ленты
Общерекурсивные функции Гёделя	композиция и минимизация вместо переходов

Ни один из трёх формализмов не знает о двух других, а класс функций у них общий. Совпадение и служит главным свидетельством в пользу тезиса.

Гёделя убедила именно механическая наглядность МТ.

Лента и клетки — это буквально человек с бумагой.

Позже тот же класс дали клеточные автоматы и другие модели, счёт формализмов растёт, класс один.

Универсальная машина

> *Можно изобрести одну машину, которой можно пользоваться для вычисления любой вычислимой последовательности.*

— Алан Тьюринг

Сколько существует машин?

Каждая МТ — конечная строка символов. Её таблица переходов записывается текстом.

Описаний длины не более n символов конечное число, а всякое описание имеет какую-то длину, поэтому описаний счётное число — счётное объединение конечных множеств. Для масштаба хватит арифметики. При алфавите из 8 символов описаний длины ровно 60 не больше $8^{60} < 10^{55}$.

Каждая машина задаёт один язык, поэтому распознаваемых языков не больше, чем машин. Языков же столько, сколько подмножеств множества слов.

Машин $\aleph_0$, а языков $2^{\aleph_0}$ — континуум.

Значит, почти все языки не распознаваемы никакой машиной.

Невычислимого неизмеримо больше, чем вычислимого.

Диагональная функция

Счётность показывает, что невычислимых функций много. Диагональ показывает, что полного вычислимого списка всех всюду определённых вычислимых функций не существует.

Предположим, что все всюду определённые вычислимые функции удалось вычислимо перечислить. Обозначим их $f_1, f_2, \ldots$, а M_n — программа, вычисляющая f_n. Пять программ из списка дают такую таблицу.

Программа	$n = 1$	$n = 2$	$n = 3$	$n = 4$	$n = 5$
M_1: всегда 1	1	1	1	1	1
M_2: n	1	2	3	4	5
M_3: $2n$	2	4	6	8	10
M_4: n^2	1	4	9	16	25
M_5: $n + 2$	3	4	5	6	7

Функция вне списка

Диагональ даёт значения $f_1(1) = 1$, $f_2(2) = 2$, $f_3(3) = 6$, $f_4(4) = 16$, $f_5(5) = 7$. Положим

$$g(n) = f_n(n) + 1.$$

Функция g всюду определена и вычислима. По n находим f_n и вычисляем $f_n(n)$. В точке n значения g и f_n расходятся, поэтому $g \neq f_n$ при всех n. Значит, никакая вычислимая нумерация не содержит все всюду определённые вычислимые функции.

Как бы ни нумеровали всюду определённые вычислимые функции, диагональ даёт функцию вне списка.

Приём тот же, что у Кантора с действительными числами, только вместо цифр стоят значения программ.

Кодирование машины

Таблица переходов — конечный текст. Состояния, символы и направления нумеруются, и каждый переход становится пятёркой (q, a, q', b, d).

Проведём через код машину инверсии битов. У неё одно рабочее состояние q_0, одно принимающее q_{accept} и три правила.

Состояния получают номера $q_0 \to 1$ и $q_{\text{accept}} \to 2$, символы — $0 \to a$, $1 \to b$, пробел $\to c$, направления — $L \to l$ и $R \to r$.

Пример (Таблица переходов в пятёрках)

Правило	Код
$(q_0, 0) \to (q_0, 1, R)$	$(1, a, 1, b, r)$
$(q_0, 1) \to (q_0, 0, R)$	$(1, b, 1, a, r)$
$(q_0, \text{пробел}) \to (q_{\text{accept}}, \text{пробел}, R)$	$(1, c, 2, c, r)$

Код как строка

Каждая строка таблицы уже пятёрка, и вся машина собирается в одну строку без пробелов.

$$\langle M \rangle = (1, a, 1, b, r)(1, b, 1, a, r)(1, c, 2, c, r),$$

Запятые делят код на поля, а скобки — на пятёрки. Чтение не оставляет неоднозначности и при двузначных номерах состояний.

Номер 1 всегда начальное состояние, номер 2 — принимающее, поэтому роли состояний в коде отдельно не записываются.

Порядок пятёрок не важен, переход ищется по паре «состояние и символ».

Код ложится на ленту как данные, а от входного слова его отделяет символ ♯, которого нет ни в коде, ни во входе.

> Программа — те же символы, которые машина читает как данные.
>
> Различие программы и данных — соглашение о роли строки.

Теорема об универсальной машине

Код машины — строка, а машина умеет читать строки. Значит, одна машина может исполнять все.

Теорема 2 (Универсальная МТ)

Существует машина U, которая по записи $\langle M \rangle \sharp w$ симулирует работу M на w.

U принимает, отвергает и зацикливается ровно тогда, когда это делает M.

Доказательство

Построим симулятор на трёх дорожках.

Дорожки. Первая хранит код $\langle M \rangle$, вторая — ленту M, третья — номер текущего состояния и метку позиции головки.

Шаг универсальной машины

Доказательство (продолжение)

Шаг. U читает состояние с третьей дорожки и символ под меткой со второй, ищет в коде пятёрку с этой парой, записывает новый символ, сдвигает метку и меняет состояние.

Остановка. Переход в состояние 2 останавливает U с той же лентой, какую оставила бы M.

Построение U с фиксированным числом состояний — работа громоздкая, но принципиально та же, что написание эмулятора на ассемблере.

Прогон на конкретном коде

Симуляция на коротком слове помещается в таблицу целиком.

Пример (Шаги U на паре $\langle M\rangle\sharp 001$)

На ленте лежит $\langle M\rangle\sharp 001$, слово записано буквами кода, $0 = a$ и $1 = b$.

Шаг	Лента M	Состояние	Что делает U
0	aab	1	ищет пятёрку $(1, a)$
1	bab	1	$(1, a, 1, b, r)$ пишет b
2	bbb	1	та же пятёрка на следующем a
3	bba	1	$(1, b, 1, a, r)$ пишет a
4	bba	2	$(1, c, 2, c, r)$ ведёт в 2

На ленте осталось bba, то есть 110, за те же четыре шага, что делает сама M.

U ищет пятёрку в коде, поэтому каждый её шаг дороже шага M.

Хранимая программа

Универсальная машина Тьюринга появилась в 1936 году.

Архитектура фон Неймана, где программа и данные живут в общей памяти, появилась в 1945 году.

> В телевизоре и в стиральной машине стоит однотипный микроконтроллер, и различает их только программа.
>
> Одна машина исполняет любую программу — в этом замысел универсальной машины.

Точная связь между работами Тьюринга и фон Неймана остаётся предметом спора историков. Общая идея при этом одна — программу можно положить рядом с данными.

Интерпретаторы

Универсальная МТ — прообраз интерпретатора. Интерпретатор читает и исполняет другие программы.

- виртуальные машины — Java, .NET, WebAssembly
- эмуляторы процессоров
- шеллы и REPL

Каждый интерпретатор держит чужую программу как данные и крутит цикл «прочитать шаг — исполнить шаг», ровно конструкция U.

> Компилятор переводит текст заранее, интерпретатор исполняет на лету, и оба читают программу как данные.
>
> Разница в моменте исполнения, а не в силе.

Куайн

Раз программы суть строки, найдётся машина, печатающая собственный текст. Такая программа называется *квайном* (*quine*).

Пример (Шаблон и подстановка)

Лобовая попытка вписать в программу её текст буксует, потому что копия обёрнута в команду печати и короче целого. Выход в том, чтобы разделить роли. Данные хранят шаблон с маркером, а код печатает шаблон с вклеенной цитатой.

```
s = "s = <>\nprint(subst(s, quote(s)))"
print(subst(s, quote(s)))
```

Цитата `quote(s)` — тот же шаблон в кавычках. Подстановка вклеивает её вместо маркера `<>`, и на печать выходят в точности две строки программы.

Дырка в шаблоне одна, поэтому текст собирается при печати, а не хранится.

Неподвижная точка описания

Самоссылка работает не в одном языке, а в любой модели вычислений.

Почему квайн не трюк?
За конструкцией стоит теорема Клини о рекурсии.

Для всякой вычислимой трансформации описаний t найдётся машина P, которая ведёт себя так же, как $t(\langle P \rangle)$.

Квайн получается при t, которая по коду $\langle M \rangle$ строит машину, печатающую $\langle M \rangle$. Неподвижная точка такой трансформации печатает собственный код.

Тем же механизмом самоссылки держится диагональное доказательство неразрешимости. Машина там спрашивает решатель о собственном описании.

Программы о программах

Программа, анализирующая программу, — обычное дело.

- антивирус решает, опасен ли файл
- статический анализатор ищет ошибки без запуска
- верификатор доказывает свойства кода

Вопрос «что сделает эта программа с этой?» стоит в центре теории вычислимости.

Универсальная машина отвечает на него исполнением. Предсказание без исполнения она не даёт.

Что сделает машина с входом?

Точная постановка задачи идёт через пару из кода и входа, $(\langle M \rangle, w)$.

Возможных исходов три. Машина M принимает w, отвергает или зацикливается, и задача в том, чтобы предсказать исход по паре.

U умеет исполнять программу. Она запускает симуляцию и повторяет шаги M.

Уметь исполнять — ещё не уметь предсказывать. Пока M не остановилась, симуляция не отличает «ещё работает» от «не остановится никогда».

Для остановившихся машин ответ приходит сам, а вся трудность сидит в зацикливании.

> Предсказателя, который отвечает на этот вопрос всегда и верно, не существует.
>
> Диагональный аргумент Кантора возвращается против самих машин.

Итоги универсальной машины

Варианты машины эквивалентны, и тезис закрепил роль меры вычислимого.

Универсальная машина превратила программы в данные и подвела к теории неразрешимости.

- Варианты машины симулируются базовой. Сила у них одна, а разное только время.
- Тезис Чёрча–Тьюринга — принятое определение вычислимости, а не теорема. Одна из его сторон неформальна.
- Машин счётное число, функций и языков — континуум. Диагональ предъявляет функцию вне любого списка программ.
- Универсальная машина исполняет любую программу. Кодирование даёт строку пятёрок, а программа — данные.
- Исполнение — ещё не предсказание.

Вопросы по универсальной машине

об универсальной машине

1. Почему многоленточная МТ не сильнее одноленточной и откуда квадратичный множитель?
2. Почему обход дерева конфигураций идёт в ширину и сколько ветвей ждёт обход на глубине 10?
3. Сформулируйте тезис Чёрча–Тьюринга и объясните, почему он не теорема.
4. У машины пустая таблица переходов и $q_0 = q_{\text{accept}}$. Сколько пятёрок в её коде и что она делает на любом входе?
5. Почему машин счётное число и как диагональ строит функцию вне списка?
6. Проследите шаги U на коде машины инверсии и входе 01.

Что почитать?

Кодирование и универсальная машина — последний материал главы m25.

Чтение закрывает варианты машин, тезис и устройство U.

Задача	Где читать
Варианты машин	m25, раздел «Устойчивость и варианты»
Тезис	m25, раздел «Тезис Чёрча–Тьюринга»
Универсальная машина	m25, раздел «Универсальная машина Тьюринга»